endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 9

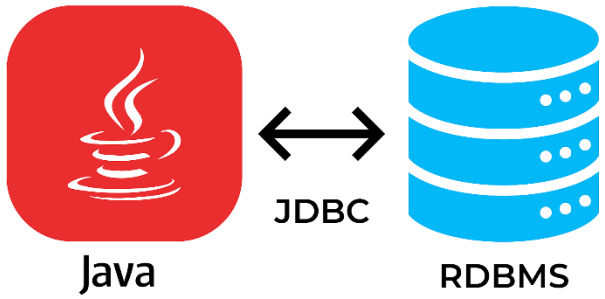
AGENDA CURS 9

1. Conectarea unei aplicatii Java la o baza de date SQL - JDBC
2. Arhitectura JDBC
3. Stabilirea unei conexiuni
4. Lucrul cu queries
5. Inchiderea resurselor JDBC
6. Managementul tranzactiilor

JDBC

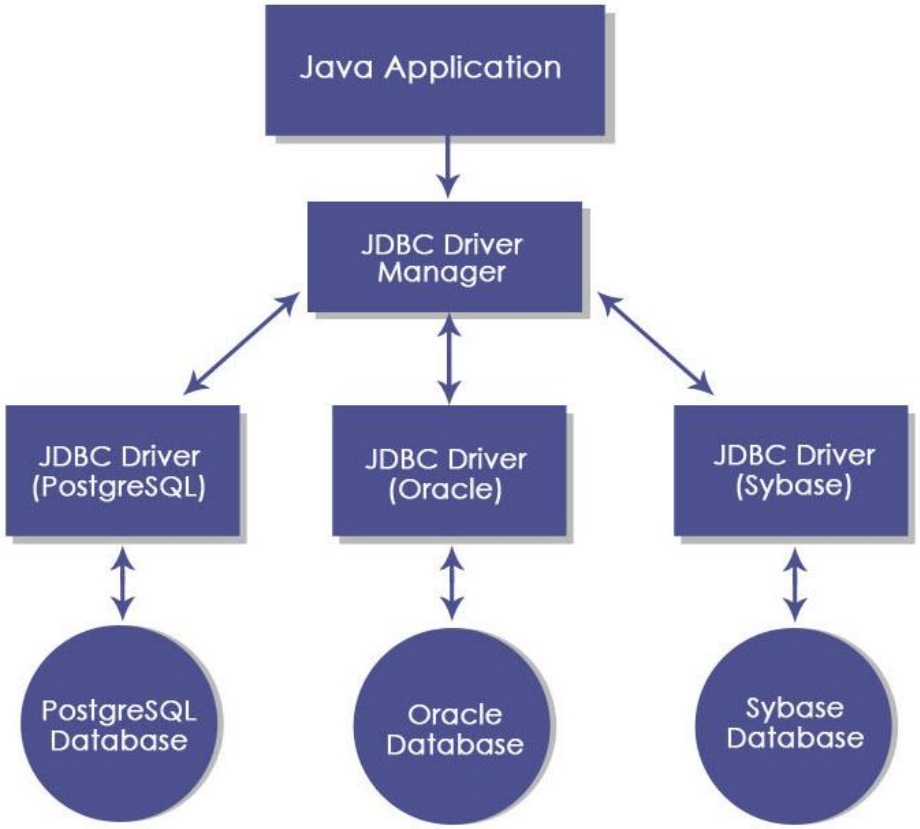
- Exista doua moduri principale de a accesa o baza de date relationala din Java:
  - Java Database Connectivity (**JDBC**): acceseaza datele ca randuri si coloane
  - Java Persistence API (**JPA**): acceseaza datele prin obiecte Java folosind un concept numit object-relational mapping (ORM)

- JDBC iti permite sa construiesti SQL queries si sa le inserezi in apeluri ale API-ului Java.



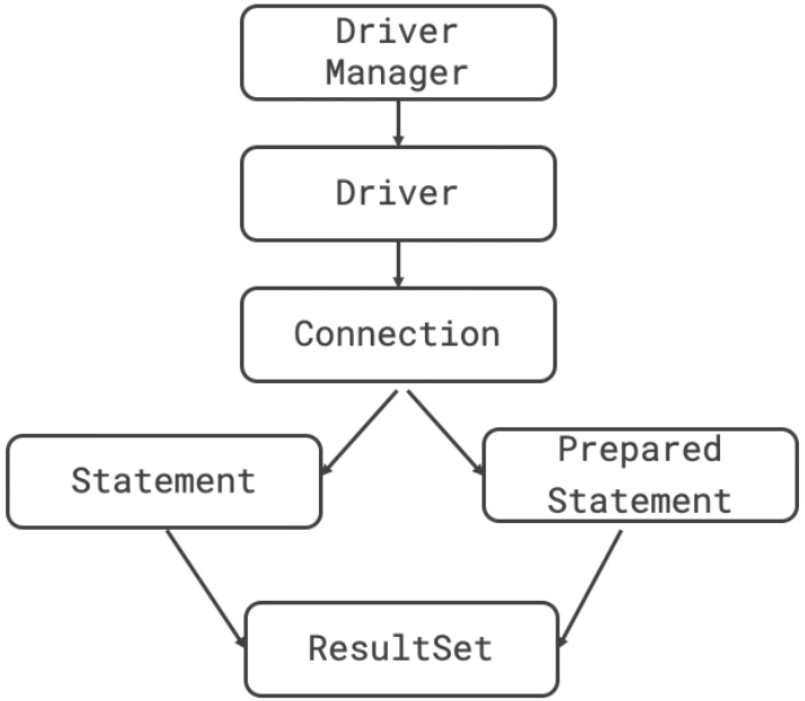
ARHITECTURA JDBC

- Componenta principala a JDBC este **JDBC Driver**-ul**.**
- Un JDBC Driver-ul este o componenta software care permite aplicatiilor Java sa se conecteze si sa interactioneze cu o baza de date. El traduce apelurile JDBC in comenzi care pot fi intelese de SGBD.
- Un JDBC Driver este format dintr-o colectie de clase si interfete care stiu sa lucreze cu un anumit SGBD.
- Exista un JDBC Driver pentru fiecare SGBD cunoscut (Oracle, PostgreSQL, MySQL, DB2 etc.).



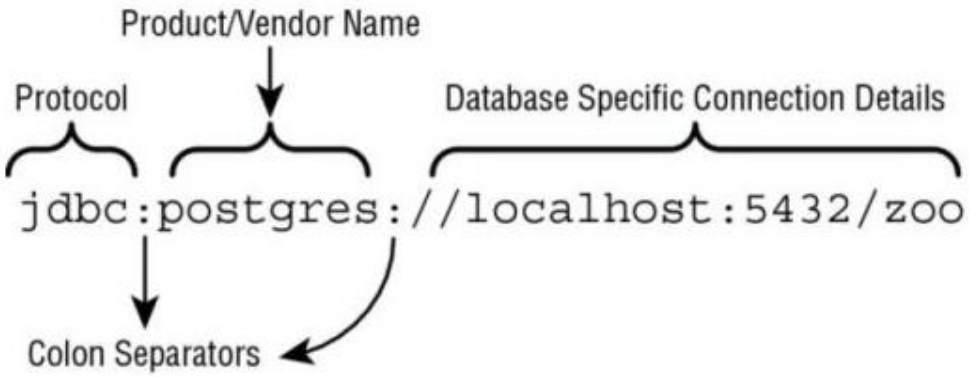
ARHITECTURA JDBC

- Interfetele din JDBC care definesc contractul pentru a lucra cu o baza de date sunt:
  - **Driver**: stabileste conexiunea la baza de date
  - **Connection**: trimite queries la baza de date
  - **Statement**: executa un SQL query
  - **PreparedStatement**: executa un SQL query
  - **CallableStatement**: executa apeluri pentru proceduri si functii stocate in baza de date
  - **ResultSet**: citeste rezultatele unui SQL query
- Toate aceste interfete se gasesc in pachetul java.sql.
- **DriverManager**: este o clasa cu rolul de a incarca JDBC Driver-ul intr-o aplicatie Java.

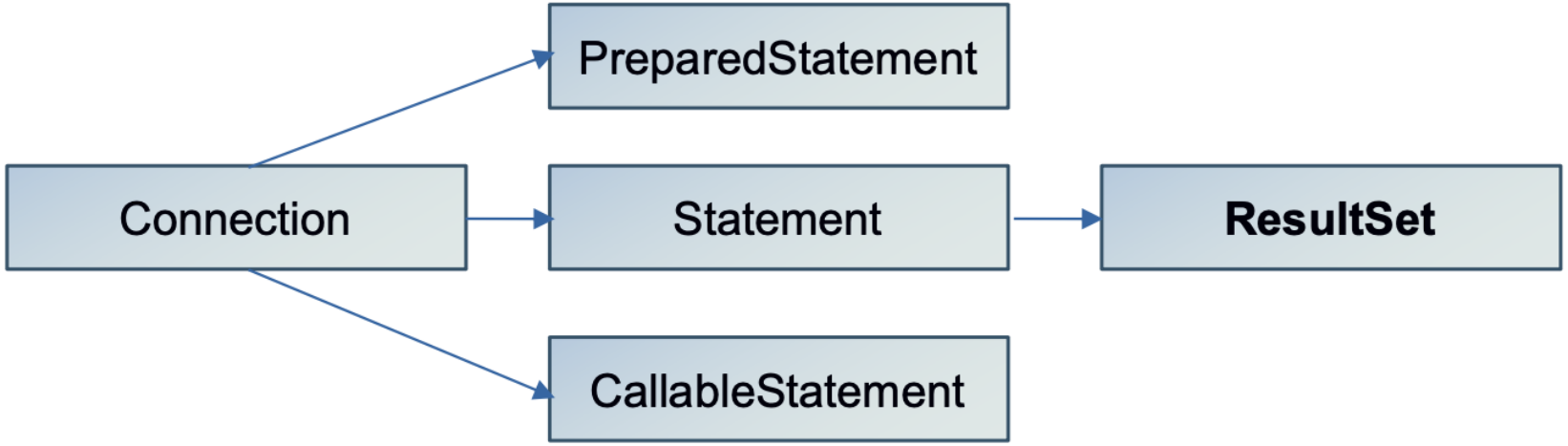


STABILIREA UNEI CONEXIUNI

- Pasii pentru stabilirea unei conexiuni prin JDBC:
  - Definirea URL-ului de conectare
  - Crearea unei conexiuni cu ajutorul DriverManager
- URL-ul de conexiune are urmatoarele componente:
  - **Protocolul**: jdbc
  - **Vendorul de baze de date** pentru care stabilim conexiunea: postgresql, mysql, etc.
  - **IP-ul si port-ul serverului de baze de date** la care se face conexiunea (port-ul poate fi omis, daca este cel default)
  - **Numele bazei de date** la care se face conexiunea



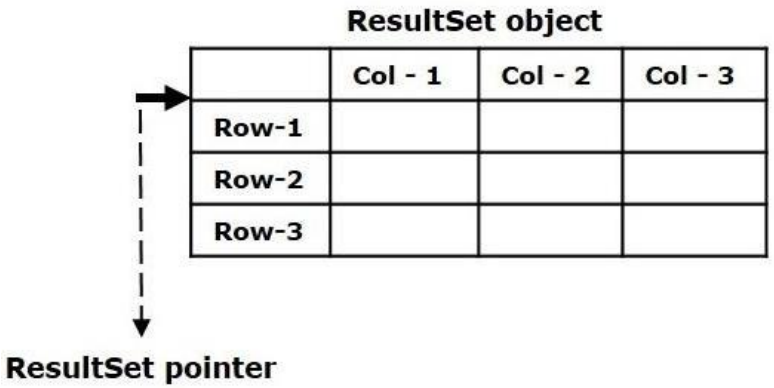
LUCRUL CU QUERIES



endava

LUCRUL CU QUERIES

- Statement si PreparedStatement: sunt folosite pentru a executa queries
- Recomandarea este de a folosi **PreparedStatement**:
  - Performanta: stabileste un plan pentru a executa eficient un SQL query si il retine
  - Securitate: protejeaza impotriva unui tip de atac numit SQL injection
  - Claritate: nu este nevoie sa folosesti concatenarea de siruri de caractere pentru a construi un query cu multi parametri
- Transmiterea dinamica de valori in queries se numeste **parameter binding** si se poate face in 2 moduri:
  - Prin pozitia parametrului (index)
  - Prin numele parametrului – recomandat
- **ResultSet**:
  - Furnizeaza o colectie cu toate records returnate de un select query.
  - Poti itera cu un loop prin toate records returnate de un ResultSet
  - Pentru un record, poti accesa valoarea unei field din select prin indexul fieldului sau prin numele fieldului



LUCRUL CU QUERIES

- **CallableStatement**: este folosit pentru a executa proceduri si functii stocate in baza de date
- Permite atat parametri de intrare (IN), cat si de iesire (OUT) sau combinati (INOUT)

```
CallableStatement stmt = conn.prepareCall("{ call procedura_nume(?, ?, ?) }");

// Setare parametri de intrare (IN)
stmt.setString(1, "valoare1");
stmt.setInt(2, 100);

// Inregistrare parametru de iesire (OUT)
stmt.registerOutParameter(3, java.sql.Types.DOUBLE);

// Executare procedura
stmt.execute();

// Preluare rezultat OUT
double rezultat = stmt.getDouble(3);
```

INCHIDEREA RESURSELOR JDBC

- Resursele JDBC sunt costisitoare de creat. Daca nu le inchizi, se produce un resource leak, care va incetini aplicatia in timp.
- Resursele trebuie inchise in ordinea inversa crearii lor.
- Pentru Statement, PreparedStatement si ResultSet, foloseste intotdeauna **try-with-resources**.
- Pentru Connection, daca e nevoie de managementul explicit al tranzactiilor, foloseste un try-catch-finally, iar in finally asigura-te ca apelezi connection.close().
- Inchiderea unei resurse JDBC ar trebui sa inchida si resursele pe care le-a creat:
  - Inchiderea unui **Connection** inchide si **PreparedStatement / CallableStatement** si **ResultSet**
  - Inchiderea unui **PreparedStatement / CallableStatement** inchide si **ResultSet**

endava

MANAGEMENTUL TRANZACTIILOR

- O **tranzactie** este o unitate logica de lucru care cuprinde unul sau mai multe operatii asupra unei baze de date (de exemplu: INSERT, UPDATE, DELETE).
- Proprietatile tranzactiilor – **ACID:**
  1. **Atomicitate** – toate operatiile din tranzactie sunt efectuate complet sau deloc.
  2. **Consistenta** – baza de date trece dintr-o stare valida in alta stare valida.
  3. **Izolare** – tranzactiile ruleaze independent unele de altele.
  4. **Durabilitate** – modificarile facute de o tranzactie confirmata (committed) sunt permanente.
- Operatii principale:
  - BEGIN TRANSACTION: incepe o tranzactie
  - COMMIT: confirma modificarile
  - ROLLBACK: anuleaza modificarile in caz de eroare

MANAGEMENTUL TRANZACTIILOR

- In JDBC, conexiunile folosesc **auto-commit** implicit, adica fiecare SQL query este tratat ca o tranzactie individuala si este committed automat.
- Pentru a face un management manual al tranzactiilor, cand vrem sa executam mai multe queries in aceeasi tranzactie, trebuie sa apelam metodele din **Connection**:
  - **setAutoCommit(false)**: dezactiveaza commit-ul automat
  - **commit()**: confirma toate modificarile
  - **rollback()**: anuleaza toate modificarile
  - **close()**: inchide conexiunea
- Inchiderea conexiunii face rollback automat daca commit-ul nu a fost apelat.

```java
Connection conn = DriverManager.getConnection(...);
conn.setAutoCommit(false); // dezactiveaza commit-ul automat

try {
    // operatii SQL
    conn.commit(); // confirma toate modificarile
} catch (Exception e) {
    conn.rollback(); // revine la starea initiala
}
```

Q&A

THANK YOU

DANIELA SPILCĂ